nature machine intelligence

Article

https://doi.org/10.1038/s42256-025-01003-z

Scalable and robust DNA-based storage via coding theory and deep learning

Received: 12 June 2024

Accepted: 23 January 2025

Published online: 21 February 2025

Check for updates

Daniella Bar-Lev [1,3] ✉, Itai Orr [1,2,3] ✉, Omer Sabary [1,3] ✉, Tuvi Etzion [1] & Eitan Yaakobi [1]

The global data sphere is expanding exponentially, projected to hit 180 zettabytes by 2025, whereas current technologies are not anticipated to scale at nearly the same rate. DNA-based storage emerges as a crucial solution to this gap, enabling digital information to be archived in DNA molecules. This method enjoys major advantages over magnetic and optical storage solutions such as exceptional information density, enhanced data durability and negligible power consumption to maintain data integrity. To access the data, an information retrieval process is employed, where some of the main bottlenecks are the scalability and accuracy, which have a natural tradeoff between the two. Here we show a modular and holistic approach that combines deep neural networks trained on simulated data, tensor product-based error-correcting codes and a safety margin mechanism into a single coherent pipeline. We demonstrated our solution on 3.1 MB of information using two different sequencing technologies. Our work improves upon the current leading solutions with a 3,200× increase in speed and a 40% improvement in accuracy and offers a code rate of 1.6 bits per base in a high-noise regime. In a broader sense, our work shows a viable path to commercial DNA storage solutions hindered by current information retrieval processes.

There is unsustainable growth in the global data sphere, fuelled by the proliferation of digital technologies such as artificial intelligence, the Internet of Things, widespread internet connectivity and the growing number of interconnected devices. While the global data sphere is anticipated to reach 180 zettabytes by 2025, current storage solutions are not expected to scale at nearly the same pace owing to capacity limitations[1]. To address this urgent need of the digital age, researchers are turning to innovative solutions such as DNA-based storage, recognizing its potential to revolutionize long-term data storage capabilities by offering extraordinary data density and durability[2,3].

A DNA molecule consists of four building blocks called nucleotides: adenine (A), cytosine (C), guanine (G) and thymine (T). A DNA strand, or oligonucleotide, is an ordered sequence of nucleotides, represented as a string over the alphabet {A,C,G,T}. The ability to chemically synthesize almost any possible strand makes it possible to store digital data on DNA molecules.

The standard in vitro DNA-based storage pipeline consists of several steps and is shown in Fig. 1a. First, the binary data are encoded into sequences over the DNA 4-ary alphabet, which are referred to as encoded sequences. Next, the encoded sequences are synthesized by a DNA synthesizer. Since current synthesis technologies cannot produce one single strand per sequence, multiple DNA strands (known as oligos) are produced per encoded sequence. Moreover, the length of the strands produced by the synthesizer is typically bounded by roughly 200–300 nucleotides to sustain an acceptable error rate[4]. The synthesized strands are then stored in a storage container in an unordered manner. To access the data, a sample of the strands is taken from the storage container, amplified using PCR and then sequenced by a DNA sequencer. The sequencer processes the strands and generates

[1]Computer Science Faculty, Technion–Israel Institute of Technology, Haifa, Israel. [2]UVeye Ltd., Tel Aviv, Israel. [3]These authors contributed equally: Daniella Bar-Lev, Itai Orr, Omer Sabary. ✉e-mail: daniellalev@cs.technion.ac.il; itaiorr@gmail.com; omersabary@cs.technion.ac.il

https://doi.org/10.1038/s42256-025-01003-z

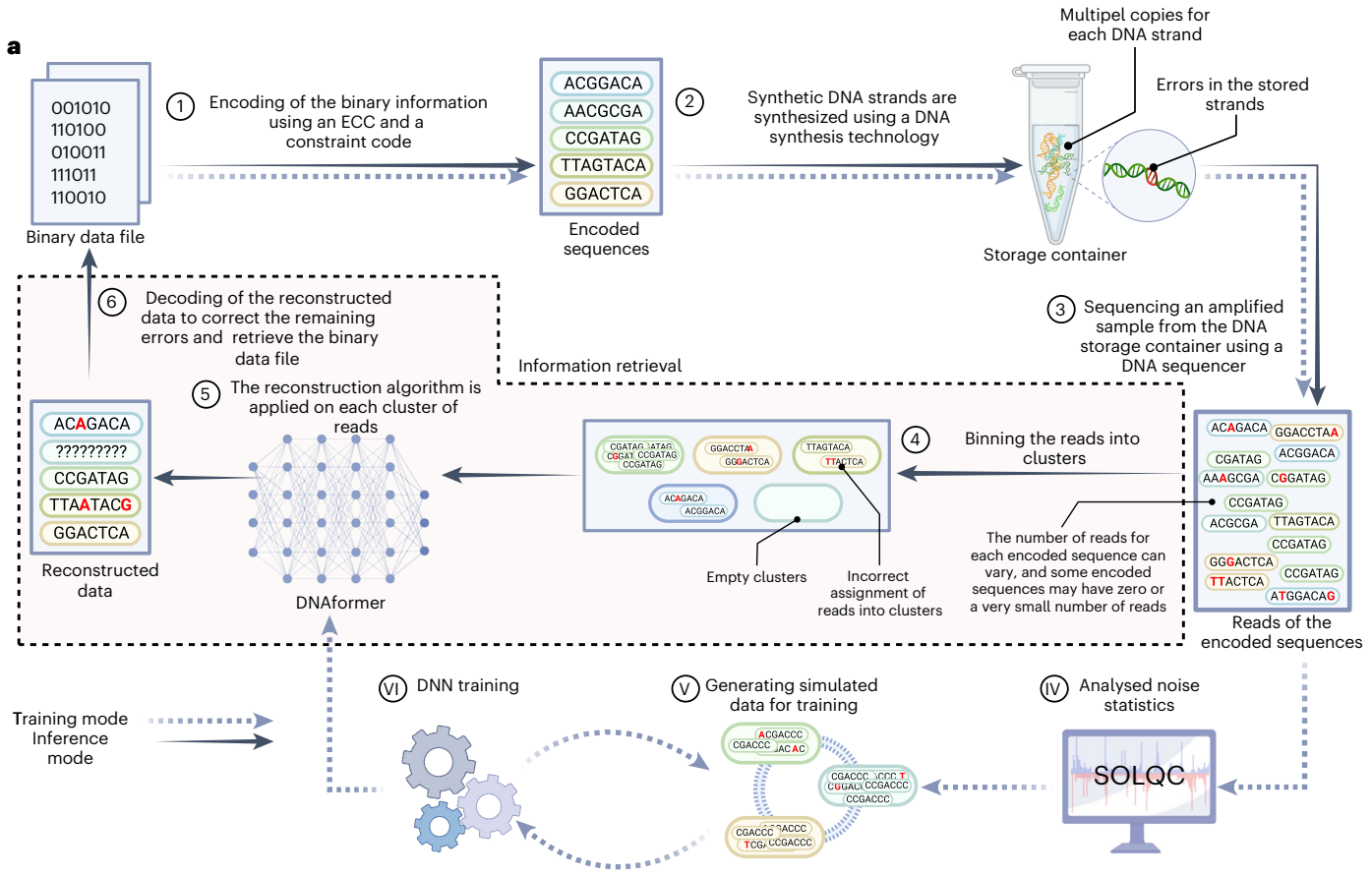


Fig. 1 | End-to-end solution for DNA information retrieval. a, A schematic description of our solution for the DNA-based storage pipeline. The different stages through the process are labelled 1–6 and steps that are part of the training pipeline are labelled iv–vi. **b**, A detailed view of the information retrieval process showing the DNN architecture, confidence filer, CPL and the input to the decoder. Figure created with BioRender.com.

reads, which are digital representations of the strands as sequences over the DNA alphabet. However, at this stage, the data are mixed in an unordered and random manner. Recovery of the original binary information is then obtained by a computationally heavy step, which is referred to as DNA information retrieval.

DNA as a storage medium has several unique attributes that distinguish it from its widespread digital counterparts and should be considered in the design of the information retrieval pipeline. The first attribute is the inherent redundancy obtained by the synthesis and the sequencing processes, where each synthesized DNA strand has several copies. The second is that the strands are not ordered in memory and thus it is not possible to know the order in which they were stored. The third attribute is the unique noise characteristics from both the synthesis and the sequencing processes, which introduce errors to the reads. These errors affect data integrity and are mostly of three types, insertion, deletion and substitution of symbols, where the error rates and their characteristics depend on the synthesis and sequencing technologies[5].

The information retrieval pipeline, also shown in Fig. 1a, can be partitioned into several main stages: clustering, reconstruction and decoding. First, a clustering algorithm is performed on the obtained reads. In this step, the unordered set of reads is partitioned into groups, where the goal is to partition the reads such that all the reads in each group originate from the same encoded sequence. Second, a reconstruction algorithm should be applied to each cluster to predict the encoded sequence. The clustering and reconstruction algorithms utilize the inherent redundancy of DNA synthesis and sequencing to correct most of the errors introduced by these processes, but usually not all.

In the DNA reconstruction problem[6–8], the goal is to predict an encoded sequence from a set of reads. One of the challenges in the DNA-based storage channel is that we do not necessarily have control over the size of a cluster, and it is likely that this size is smaller than the required minimum size that guarantees a successful reconstruction of the encoded sequence in the worst case. According to the classical Levenshtein reconstruction problem[9], the number of reads required in the worst case is polynomial with respect to the sequence length. In contrast, models that consider retrieval only with high probability suggest that this number is logarithmic with the sequence length[10]. While these results were obtained under several theoretical assumptions, which rarely hold in reality, they can still be considered as lower bounds when these assumptions are omitted. Lastly, the decoding step converts the reconstructed encoded sequence back to the binary data. In this step, if an error-correcting code (ECC) was applied during the encoding phase (before the DNA synthesis step), the remaining errors can be corrected using the decoding procedure of the ECC.

Church et al.[11] and Goldman et al.[12] first demonstrated large-scale in vitro DNA storage, storing 643 KB and 739 KB, respectively, though neither fully recovered their messages. Later, Grass et al.[13] used Reed–Solomon (RS) coding[14] to successfully store and recover 81 KB. Erlich and Zielinski[15] introduced DNA Fountain, recovering 2.11 MB of data. Organick et al.[16] developed a random-access DNA scheme, storing 200 MB. Yazdi et al.[17] presented a coding scheme to correct errors from Nanopore sequencer, successfully encoding 3.633 KB. Wang et al.[18] stored 379.1 KB using an inner–outer scheme combining cyclic redundancy check and repeat–accumulate code, nearing 1.69 bits per base. Chandak et al.[19] introduced a convolutional code and recurrent neural network method with Nanopore MinION, storing 11 KB. Anavy et al.[20] showed how composite letters can increase DNA storage capacity.

Channels with synchronization errors, such as insertions and deletions, present theoretical challenges that remain poorly understood. Unlike channels with erasures and standard errors, where both channel capacity and capacity-achieving codes are well defined, the theory for synchronization channels lacks these fundamental understandings. This gap in knowledge underscores the complexity and unresolved nature of addressing synchronization errors in communication and storage[21]. The latter makes the design of both clustering and reconstruction algorithms more complex[22,23]. Additionally, the clustering problem is a computationally intensive problem by itself. This is especially challenging when applied to DNA-based storage where the number of clusters can be extremely large, for example, 1 TB of data will require an order of billions of clusters. Furthermore, theoretical reconstruction algorithms designed to address deletion and insertion errors, usually assumed that the clusters were partitioned (almost) perfectly[6,24,25] or were designed to work on a large block length[24,26–29].

Several works[13,15,19] tackled these difficulties by adding redundant symbols to each designed DNA strand (that is, inner coding) or by adding redundant DNA strands (that is, outer coding) to detect and correct the deletion and insertion errors. In these techniques, the clustering and the reconstruction steps can be avoided. In this approach, the inherent redundancy of the synthesis and the sequencing processes is not fully utilized, which leads to suboptimal use of redundancy in the design of the ECC. This, in turn, can also lead to performance degradation due to an increase in the number of strands and reads that should be processed.

Machine learning methods for DNA-based storage were explored by Bee et al.[30], who proposed a content-based similarity search for integration into such systems, and by Pan et al.[31], who introduced a rewriteable DNA storage system by encoding data in both the DNA strand and backbone. Both approaches, relying on low entropy, are best suited for structured data.

End-to-end solution for DNA information retrieval

In this study, we suggest a practical, holistic and computationally scalable pipeline to storing and retrieving information for in vitro DNA-based storage systems, while addressing the various challenges mentioned above. A schematic description of our proposed pipeline is shown in Fig. 1. Our solution, termed DNAformer, utilizes a modular encoding scheme, combining ECC and constrained codes before DNA synthesis and storage. Our coding scheme enables the pragmatic partitioning of large datasets into smaller blocks to allow for fast and easy access to specific parts within the data. The exact block size is a user-defined parameter, and more details about the parameters used in our datasets can be found in the Supplementary Information. When a specific part of the data needs to be accessed, a sequencing process is used, followed by an information retrieval process. The first step in this process is to bin the different reads into groups based on their index. This naive approach introduces noise into the clusters, which we treat in the following steps. The benefit is an increase in the speed of the clustering step over alternative, more noise-tolerant approaches, such as the hash-based approach[23] and the Clover clustering approach[22].

In the second step, we utilize a deep neural network (DNN) to reconstruct a sequence over the DNA alphabet based on a non-fixed number of reads with varying lengths and solve a sequence-to-sequence, multiple-in-single-out problem. The model architecture shown in Fig. 1b uses a combination of convolutions and transformers followed by a confidence filter to screen correct predictions from false ones. The suspected incorrect predictions can go through a second reconstruction step, a dynamic programming-based algorithm, termed conditional probability logic (CPL). The CPL algorithm analyses the reads in each cluster and estimates their possible errors and probabilities according to the reads' similarity in the edit distance metric. The main advantage of the CPL algorithm is that it requires zero prior knowledge of the cluster's error probabilities, and by estimating them it is possible to predict a good estimation even for small and erroneous clusters. This step adds another degree of freedom in our solution to deal with the tradeoff between accuracy and speed where the purpose is to deal with more challenging cases while not sacrificing the run-time capabilities of the system. Furthermore, our approach uses the concept of safety margin to quantify how robust the information retrieval pipeline is under specific working conditions.

The third step is the decoding of the data, where we utilize a modified tensor product (TP) code[32]. In our solution we modify the standard TP approach to take advantage of the first two steps, the clustering and reconstruction steps, and create a coherent, unified pipeline. This approach allows us to take advantage of the inherent redundancy and success of the upstream steps to reduce the redundancy within the coding scheme. Our scheme allows simple integration with a large family of constrained codes and flexibility with error correction capabilities.

To address the high cost of acquiring large training datasets for DNNs, we used simulated data, leveraging a small amount of real data to analyse and model synthesis, PCR and sequencing error rates via the SOLQC tool[33]. Once modelled, these errors enable the generation of unlimited simulated data for DNN training. Notably, errors are modelled only once per synthesis and sequencing process, which makes our method scalable and cost effective.

A key point of our solution strategy is that we do not teach the model to utilize underlying semantics and file structure, but rather focus on the noise characteristics of the synthesis and sequencing processes. This gives the model the important ability to process unstructured and structured data with similar performance. Further details are provided in the Methods section and Supplementary Information.

DNA dataset

On the basis of our methodology, we partitioned the dataset into two parts. The first is a pilot dataset containing random information encoded into 1,000 sequences to analyse the noise characteristic, and the second is the main dataset of 110,000 encoded sequences, termed the test dataset. The two datasets were synthesized by Twist Bioscience and obtained at different dates for both the synthesis and sequencing steps to make sure that they are as independent as possible.

To examine our approach on different data modalities, the test dataset was purposefully split into two files, each containing 55,000 encoded sequences. The first is a compressed (zipped) folder with three modalities: an image, a 24-s audio snippet, taken from Neil Armstrong's iconic moon landing, and a text file from the DNA Data Storage Alliance[34]. In total, the size of this compressed file is about 1.5 MB. The second file contained about 1.5 MB of random information bits. The data are shown in Fig. 2a–d.

To examine our approach on different sequencing technologies, sequencing was done using both an Illumina miSeq and an Oxford Nanopore MinION. Sequencing with Illumina produced 528,636 reads for the pilot dataset, with an error rate of 0.079% and an average cluster size of 529. The test dataset had 3,215,249 reads, a 0.123% error rate and an average cluster size of 29.

For Nanopore sequencing, the pilot dataset yielded 2,805,705 reads, with an error rate of 4.6% and an average cluster size of 754. The test dataset was sequenced twice to ensure the average cluster size was sufficiently large. The first flowcell produced 4,341,575 reads, an error rate of 4.1% and an average cluster size of 13. The second flowcell produced 3,065,455 reads, with an error rate of 5.07% and an average cluster size of 8. This flowcell difference stems from varying flowcell quality, laboratory procedures and DNA sample. The combined two flowcells produced 7,407,030 reads, an error rate of 4.47% and an average cluster size of 21.

The SOLQC tool[33] was used to analyse the noise characteristics, showing a good fit between our pilot and test datasets (Fig. 2e,f). It is important to highlight that our suggested solution does not require an equivalence between the error distributions of the pilot and the test datasets, but only a similar behaviour in terms of order of magnitude and types of errors. As our approach is based on generating simulated data for training, this illustrates the premise that it is possible to overcome challenges such as domain transfer between the simulated data used for training and the actual data during operation based on our approach. Additional details on our datasets can be found in the Supplementary Information.

Results

To examine and compare the accuracy and performance of the DNAformer, we used both of our datasets, Illumina and Nanopore (termed after the sequencing technology used), as well as four additional publicly available datasets[7,13,15,16]. The previously published datasets differ in their synthesis and sequencing technologies, leading to different noise characteristics. To allow a fair comparison, all datasets were clustered using our binning approach. Since some of the datasets do not include indices in their encoded sequences, we used the first unique symbols of these sequences as if they were indices. Additional details on the publicly available datasets used can be found in the Supplementary Information.

We compared our method with five published algorithms. The first three use a symbol-wise majority vote to predict the most frequent symbol at each read position: BMA Lookahead[35], Divider BMA[6] and the Viswanathan and Swaminathan (VS) algorithm[24], all based on bitwise majority alignment (BMA)[25]. The fourth, the iterative algorithm[6] uses dynamic programming to detect and concatenate frequent subsequences. The fifth is a hybrid of the iterative and divider BMA algorithms, based on cluster error probability. A comparison with the trellis-based method from Srinivasavaradhan et al.[7] is provided in the Supplementary Information.

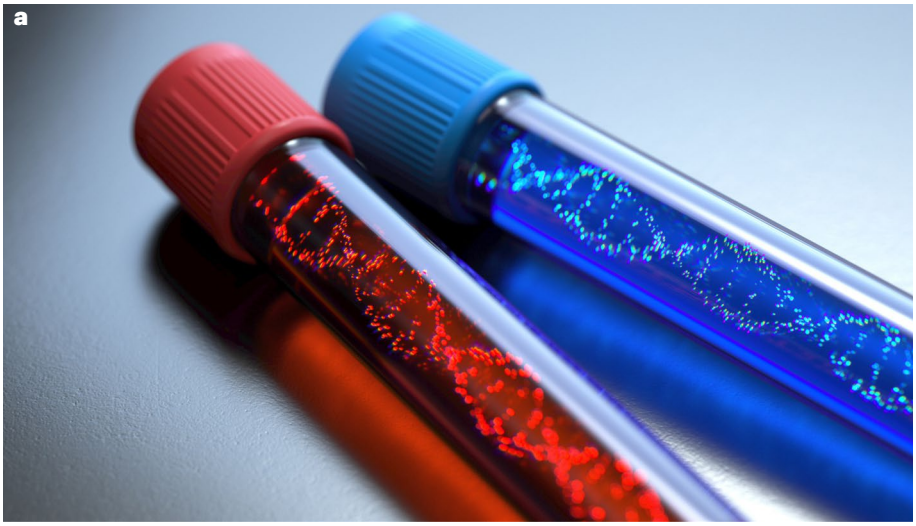
Notable work by Yazdi et al.[17] presented a coding scheme to correct errors caused by Nanopore sequencers and relied on coding constraints (such as exactly 50% of GC content). As these constraints are not satisfied in our datasets, we did not include this method in our report.

A comparison of our approach with leading reported methods is provided in Fig. 3a, where the DNAformer achieves state-of-the-art (SOTA) results. On our Illumina and Nanopore datasets, the DNAformer achieves a failure rate of 0.0055% and 1.65%, respectively. Additionally, our method achieves a failure rate of 0.02%, 0.66%, 0.17% and 14.58% on the datasets provided by Erlich et al.[15], Grass et al.[13], Organick et al.[16] and Srinivasavaradhan et al.[7], respectively. We note that the closest method in reconstruction ability to the DNAformer is the iterative method[6].

We further compare the accuracy of the DNAformer on different data modalities in our dataset. In this context, the accuracy is defined as the fraction of correct predictions out of the total number of clusters obtained from our binning step. The first file includes text, an audio message and a photo, while the second is a file consisting of random bits. Our results, provided in the Supplementary Information, show similar accuracy for both files, demonstrating that our method does not rely on the underlying semantics or structure in the data, but rather on the noise characteristics from the synthesis and sequencing processes. In addition to the reconstruction ability, it is also important to examine the inference speed of the different methods, as storage-based applications greatly favour fast processing speeds. The results are reported in Fig. 3b, where we compare the time each method will take to reconstruct 100 MB of data, under the assumption that the information was encoded using our suggested coding scheme. Our method greatly improves on current methods by several orders of magnitude. When compared with the second-in-performance method (that is, the iterative method), we improve the speed by 3,200×. Therefore, DNAformer simultaneously improves both the reconstruction accuracy and speed, without the natural tradeoff that usually exists between the two.

To guarantee the retrieval of binary data, our method uses a TP-based, modular coding scheme. When comparing different coding schemes, an important parameter to consider is the code rate, which signifies how efficient the code is in terms of redundant symbols. Furthermore, the code rate needs to be evaluated alongside the channel's error probability, as an increase of this parameter makes the retrieval more challenging. Figure 4a shows a comparison between different coding schemes, where we see our work achieves a high code rate in addition to being utilized in a relatively high-noise regime.

https://doi.org/10.1038/s42256-025-01003-z



d

"Information is being digitized on a massive scale, by servers in datacenters, by mobile devices, and by networks of sensors everywhere around us. Artificial intelligence techniques and ubiquitous processing power are making it possible to mine this massive ocean of data; however, integral to harnessing this data as knowledge is the ability to store it for long periods of time. Legacy storage solutions have scaled extensively over the years, but the areal density of magnetic media (HDD and tape), which enables today's mainstream archival storage solutions, is slowing, and the size of libraries is becoming unwieldy. In short, data growth is outpacing the scalability of today's storage solutions. The industry needs a new storage medium that is more dense, durable, sustainable, and cost effective in order to cope with the expected future growth of archival data.
DNA, nature's data storage medium, enters this picture at a time when synthesis and sequencing technologies for advanced medical and scientific applications are enabling the manipulation of synthetic DNA in ways previously unimagined. There are credible predictions that, by 2030, DNA synthesis could reach a cost of $1/terabyte and that DNA sequencing may reach similar levels. The scale of DNA data storage is unprecedented; the volume of space inside an LTO tape cartridge is estimated to hold 100,000 times the number of DNA-bits as an LTO-9 tape in that same cartridge. The durability of DNA and uniformity of the DNA molecular structure are ideally suited to long-term archival storage. Finally, DNA is an inherently environmentally friendly medium in terms of power, space, and sustainability, which will place significantly lower burdens than legacy storage technologies on our fragile ecosystem.
The intersection of the "data overwhelm" and our ability to manipulate synthetic DNA offers a vision of a new layer in the storage hierarchy that could radically change the scale of what we store and how long we store it. Preserving our digital legacy in turn opens possibilities to extract, and even create or discover, new knowledge."

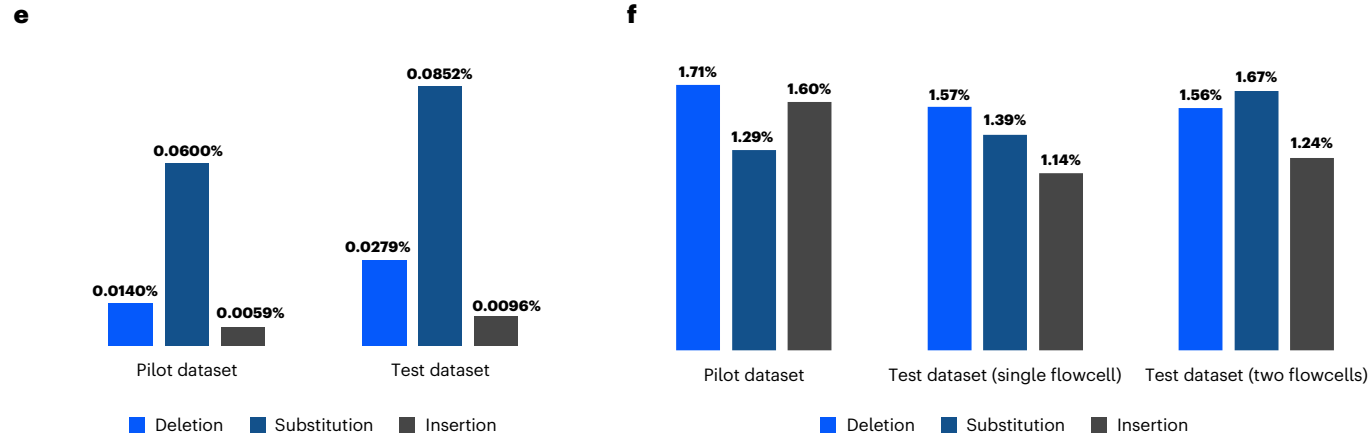


Fig. 2 | Data used for DNA experiments. a–**d**, The data as an image (**a**), audio file (**b**), random bits (**c**) and text (**d**). **e**,**f**, Statistical analysis of the errors found in the Illumina (**e**) and Nanopore (**f**) datasets. We deliberately chose these data modalities to examine the performance of our method under different conditions.

Apart from guaranteeing information retrieval, the coding scheme presented in this work also introduces safety margins, which measure how robust is the DNAformer under specific working conditions. As there are three main categories of errors from DNAformer, where some are more difficult to correct than others, our code has been designed to support a range of values for each error type. Figure 4b shows a heat map with the three types of errors the code can correct, and α, β and γ denote the quantities of each type of error. We see that when tested on the Illumina dataset, DNAformer is well within the safety margins for information retrieval. When tested on the Nanopore two flowcells, DNAformer can retrieve the information; however, the safety margin has been decreased. When tested on the Nanopore single flowcell, which resulted in smaller cluster sizes, we observe that using only the DNN fails to retrieve the information. However, following the method shown in Fig. 1, by applying the safety margin mechanism with both the confidence filter and the CPL, DNAformer can successfully retrieve the information. Additional experimentation results, ablations studies and comparisons are presented in the Supplementary Information.

Discussion

When considering commercial, real-life applications of DNA-based storage, several system-related considerations arise, such as robustness, scalability, run time and compute requirements. As this work presents an end-to-end solution to the DNA information retrieval problem, several individual components in the pipeline have been redesigned. Additionally, focus was placed on the interaction between the different components which is critical to a successful operation. The results show SOTA performance in accuracy and run-time speed on several publicly available datasets in addition to new datasets provided in this work. Extensive experimentation and ablation studies are provided in the Supplementary Information, showcasing the modular nature of this approach and how it can be modified to different applications and settings.

Our solution is based on the combination of TP-based ECC wrapping a transformer-based DNN while considering the system-related considerations mentioned earlier. To overcome the run-time limitations of current DNA-based storage pipelines, while using the inherent

https://doi.org/10.1038/s42256-025-01003-z

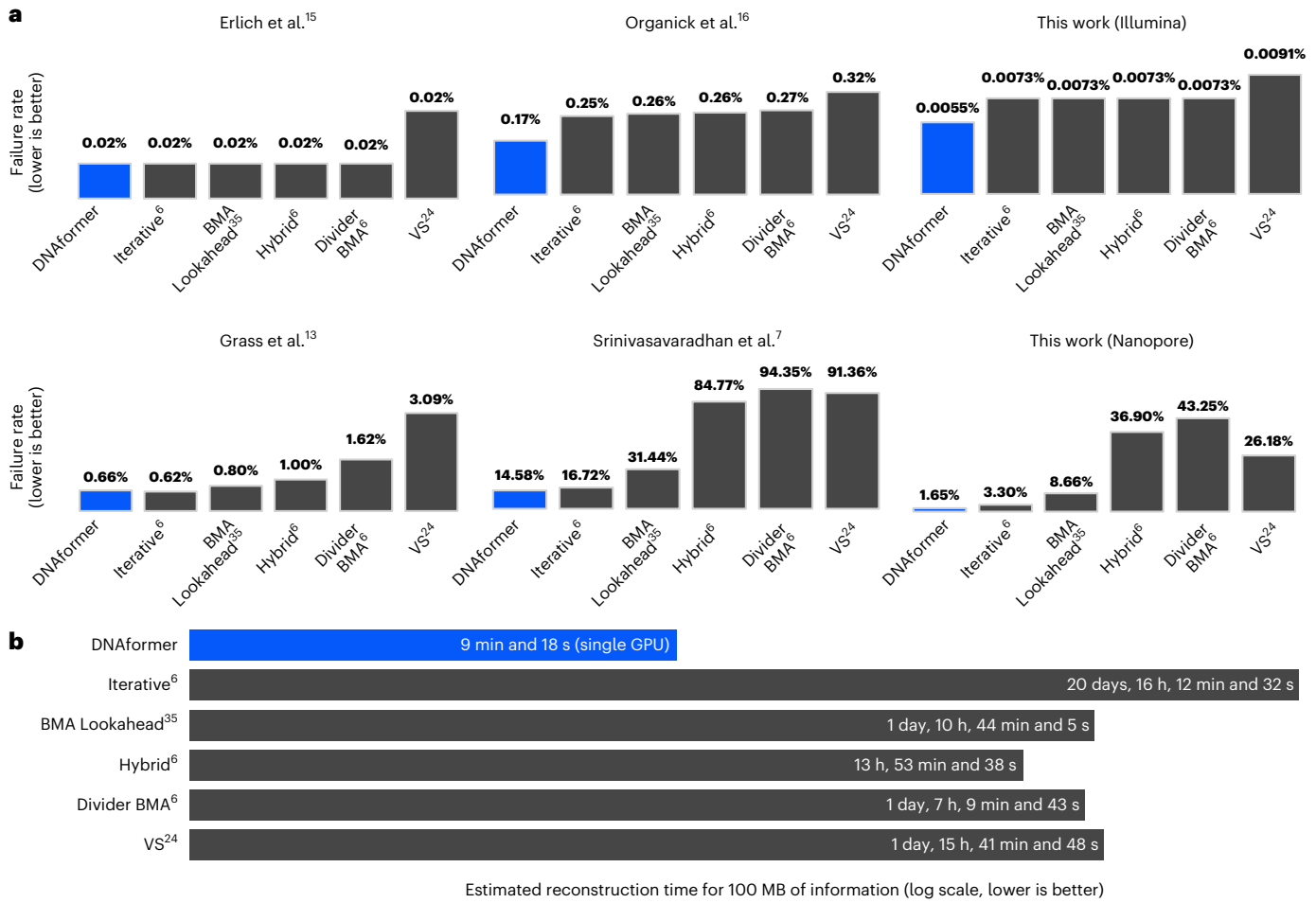


Fig. 3 | Comparison of the DNAformer with SOTA DNA reconstruction methods. a, The failure rate over several publicly available datasets as well as the Illumina and Nanopore test datasets, where the DNAformer achieves SOTA results. **b**, The estimated reconstruction for 100 MB of information, encoded using our suggested coding scheme, where we see our method achieves a reduction in the required time.

redundancy of the DNA-based storage channel, our design uses a naive and very efficient method for clustering. However, this comes at the cost of noise within each cluster. Therefore, the information retrieval pipeline needs to be able to overcome this type of noise during the reconstruction phase. Traditional reconstruction methods, which are primarily central processing unit based, often rely on dynamic programming techniques that are computationally intensive and less amenable to parallelization, making them not scalable.

DNNs are a good fit to address these challenges due to their parallel computational nature and graphics processing unit (GPU) implementations, which allowed us to achieve inference time several orders of magnitude faster than previous solutions. By leveraging the parallel processing power of GPUs and adopting a multiple input, multiple output strategy, where DNAformer predicts several tokens simultaneously, our model avoids the linear increase in inference time typical of auto-regressive predictions in other transformer-based models. Moreover, the model architecture is highly efficient, with its largest configuration comprising only 100M parameters, much smaller than current leading large language models. These design optimizations enhance both processing speed and performance, ensuring scalability and computational efficiency. However, the current cost of producing a large volume of real data for training such a model is high. In addition, since there is a need to employ an ECC before the DNA synthesis process, some changes in the design of the code will require to generate a new dataset. For these reasons, we turn to simulated data to train our model.

During training, we utilized 1.4B simulated DNA reads, dwarfing any existing datasets for DNA-based storage. The cost to create such a large dataset from real DNA (that is, not simulated data) is estimated at over US$10M, far greater than most laboratories' budget. Showing how our method paves the way for a realistic, large-scale application of DNA-based storage. Our training methodology uses a small amount of data from real experiments to model the errors, from which we can create an unlimited amount of simulated training data.

The proposed coding scheme introduces different attributes. First, we consider the entire information retrieval pipeline, which allows a simplification of the channel. Our approach strips away complexities involved with correcting insertions and deletions, making the process simpler. Second, by strategically leveraging a TP code, instead of the inner–outer code approach, we demonstrated that the integration of the clustering and reconstruction steps into the decoding process leads to a reduction in the required number of redundant symbols for error correction. Third, we offer a modular way to incorporate various constrained codes into our scheme, a known challenge in coding theory[14,36]. This allows for a versatile means of adapting to different conditions or requirements, enhancing the adaptability of our approach across various scenarios.

From a system design perspective using the confidence filter and CPL algorithm increases the safety margin and allows our method to expand its ability to solve difficult cases. For example, higher noise regimes or smaller cluster sizes, a desired trait in terms of cost and time. It is worth mentioning that the accuracy of DNAformer does not

https://doi.org/10.1038/s42256-025-01003-z

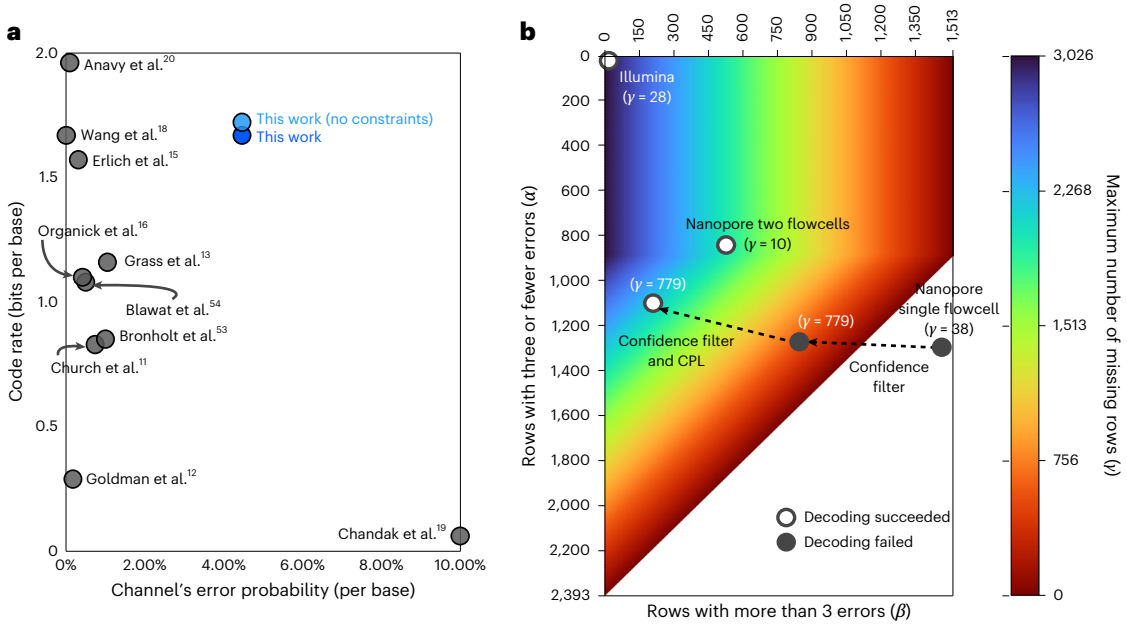


Fig. 4 | Evaluation of information retrieval performance. a, A comparison of the code rate versus the channel's error probability of different coding schemes that were used in DNA-based storage experiments. Our results show a competitive code rate even while operating in higher noise regimes. **b**, A heat map of the errors regime where our coding scheme is able to operate successfully. The specific instances of our three datasets are shown on top and illustrate the failure/success of the retrieval, as well as the associated safety margin.

depend on the coding scheme, as exemplified in Fig. 3, when evaluating DNAformer on previously published datasets. However, as exemplified in our results (Fig. 4), the integration of DNAformer with our proposed ECC allows us to reduce the coding redundancy while still allowing information retrieval.

Our proposed method of integrating ECC and constrained codes extends outside the domain of DNA-based storage and can be useful in other channels as well. Moreover, the utilization of a TP-based scheme that relies on similar ideas can be used for scenarios in which a DNN (or any other algorithm) can be applied to parts of the data pre-decoding.

Conclusions

In this Article, we present a scalable method for DNA-based storage that overcomes some of the major bottlenecks in current solutions for balancing failure rate and run time. Our method combines a DNN and ECC to leverage the sequencing and synthesis inherent redundancy without compromising on performance. Furthermore, our solution reduces the required ECC redundancy and the required number of reads for error-free recovery of the information. Our results show that DNNs can improve the decoding process in a DNA-based storage system and shorten a DNA-based storage system response time by several orders of magnitude. From a broader perspective, our DNN–ECC approach overcomes a major bottleneck on the path to large-scale commercial applications of DNA-based storage.

Methods

Our method is an end-to-end solution to the DNA information retrieval problem, as shown in Fig. 1a. As part of this approach, it was divided into several components, each with its own functionality: encoding of sequences before DNA synthesis, clustering of reads post sequencing, reconstruction and decoding back to the original data. In addition, the interplay and interface between the different components was also taken into consideration as part of a complete system-level, optimized solution. The solution combines coding theory and deep learning methods to create a holistic and coherent pipeline to encode and decode the DNA data.

One of the merits of DNA-based storage is high data density, meaning a scalable storage system needs to be able to quickly process arbitrary large files. To create a scalable method, we do not require the entire file to be processed simultaneously and design our method to process data in smaller batches, hence our solution can be adapted to random access purposes. Our suggested solution encodes a block of 1.53962 MB into 55k designed encoded sequences of length 140. Each strand is composed of 12 bases for index, including a single base that serves as a file identifier. The remainder 128 bases were allocated for the binary data and the redundancy from the ECC and constrained code. This length was chosen to allow for efficient DNN processing on a GPU during the reconstruction step and the coding scheme during the encoding and decoding steps.

Coding scheme

Our coding scheme is a modular pipeline composed of several components, each addressing a different purpose: index encoding, diagonal column encoding, constrained code and TP code. The complete encoding and decoding pipeline is designed to integrate these four components in an interleaving order that allows each of them to achieve its designed goal without hindering the others.

There are three main considerations in the design of our coding scheme. First, our solution assumes that the decoding is performed after the clustering and reconstruction algorithms. By design, these steps eliminate most of the deletions and insertions errors, and the output has the same length as the encoded sequences. Hence, after this step, we need to only take care of substitution errors and missing predictions due to either missing clusters or our confidence filter, which is easier to solve in terms of coding theory and requires less redundancy.

As seen in Fig. 3a, when the clustering and reconstruction perform well, a high number of error-free predictions can be passed to the decoder, so the decoder only needs to correct errors in a small fraction of the predicted sequences, rather than in all of them. Therefore, a main component of our code is a TP-based coding scheme that can correct up to a specified number of errors in up to a specified fraction of the predictions, allowing for a reduction in the redundancy needed to ensure error-free reconstruction of the data.

To maintain error-free retrieval of the information, our code also uses a diagonal column encoding, which serves as an outer code that can correct the remaining erroneous predictions and overcome the missing ones. This code is implemented using an RS code, which is applied on the encoded sequences that store the information. As some sequencing and synthesis technologies are more error prone at the beginning and end of each read[33,37], the RS code was designed diagonally, meaning the information at the beginning and end of each read will be encoded together with the information in the middle to create a more uniform distribution of the errors.

The second consideration in our design is that our clustering step is essentially a binning algorithm that uses the index part of the reads and matches them to the valid indices that were used in the designed sequences. Hence having errors in the index can cause a read to be ignored or misclassified. As misclassified reads are more problematic during the reconstruction step, we use an index encoding that is based on a pre-calculated set of indices that are of edit distance three or more from each other.

Our third consideration relates to constrained coding for DNA-based storage. The different sequencing and synthesis technologies lead to different constraints that should be addressed, depending on the specific technology. For example, in our datasets, we selected the mapping to avoid the long homopolymer (consecutive repetition of the same base) of length 5 or more and GC content of each encoded sequence between 45% and 55% with high probability. These constraints were selected to preserve the stability of the synthesized strands and mitigate their error rates, considering the technologies that are used in our experiments[38–40].

Additionally, recent work demonstrated that in some cases, it is better to not impose any constraint[39]. Hence, to keep our code flexible for different use cases, we designed a block-based constrained code in a way that is almost independent of other components of our scheme. This allows an easy adaptation to other constraints or a removal of the constrained code component entirely. In our design, removing the constrained code improves the information rate by 7.6%.

Clustering

From a system-level optimization perspective, this step was optimized for speed rather than accuracy. We adopt a naive clustering approach based on simple and fast binning of the reads based on their index. The goal in our design for this step is to overcome slow clustering methods[22,23] and reduce the processing time. However, this comes at the cost of noise, introduced by errors in the index of each read, which causes some of the reads to be wrongly clustered (that is, false reads) or to be dropped during the clustering step. On a system-level basis, we overcome this noise in the reconstruction step using a DNN.

Reconstruction

Designing a method that combines a DNN and ECC requires the ability to iterate between the two parts during the design phase. That is, the coding scheme and the DNN are coupled together to guarantee a specific set of success metrics. However, creating a different training dataset for each coding scheme modification is a costly and resource intensive process. For example, using previous DNA-based storage systems[15,41], the estimated cost of synthesizing 1 GB of data is roughly US$3–5M.

Owing to this fact, we turn to simulated data for training our DNN. The main challenge when using simulated data for training is the generalization to real-world settings after the model is trained. To overcome this issue, we construct a data generator based on statistics from real-world experiments[33,42]. These statistics contain the error probabilities, which are used to generate the reads of each label.

Simulated data generation. Our reconstruction approach uses a DNN that predicts a single label sequence from a cluster of reads. Since the data are randomly sampled, each cluster can vary in size and each read can vary in length due to synthesis and sequencing errors. Moreover, some clusters can suffer from higher error rates compared with others. Our method utilized simulated data only during training, meaning we did not use real data during the training of the models. Pseudo code for each iteration of the training process using our simulated data generator is given below:

(1) Draw a random sequence of letters based on the 4-ary {ACGT}.
(2) Encode the sequence using some error-correcting and constrained coding scheme (optional).
(3) Draw a random number of reads and a random number of false reads.
(4) For each read:
   (a) Draw random deviation from the modelled error probabilities. The modelled error probabilities are provided as average error rates and base-dependent error rates. The average error rates represent the probability of observing any type of error, while the base-dependent error rates represent the probability of observing a specific error event at a particular base. These error probabilities are calculated using SOLQC[33].
   (b) Inject simulated errors for each read based on the error model[42]. In the simulated reads, errors are independently injected at the base level. For each base (symbol) of the generated reads, an independent random process determines whether an error occurs based on the preset average error rates. If an error event is determined, the type of error is selected based on the observed–simulated base (symbol) and the base-dependent error probabilities. It should be noted that our model assumes the errors are close to being independent, as characterized in most of the cases shown in ref. 33 and also in our pilot dataset.
(5) Batch several clusters.
(6) Forward–backward pass through the DNN.

Data preprocessing. The DNAformer architecture processes the reconstruction of a cluster of reads, meaning a set of reads is processed simultaneously to reconstruct the suitable encoded sequence. Preparation of the reads includes filtering short and long reads beyond a specified design parameter. This ensures that highly corrupted reads will not affect the reconstruction process. Next, a simple one-hot encoding and padding are used to prepare the data to the model's encoder and make sure all reads are of the same length.

Model architecture. Our model uses a combination of convolutions and transformers. We adopt the concept of early convolutions before a transformer block to improve training stability and performance[43]. The model is structured as a Siamese network where the two branches share weights and are different by reversing the symbol order before the branch termed 'right branch' in Fig. 1b. This is done since the padding used in the preprocessing step is concatenated only at the end of each read. An alternative approach will be to align all the copies to one another using sequence alignment methods[44]. However, these methods come at the cost of 'in-series' computation and processing time and therefore we decided to avoid them.

Instead, we created an alignment module whose purpose is to learn the required alignment for each read independently. The alignment module uses an Xception[45] inspired architecture with depthwise separable convolutions and multiple kernel heads. The purpose of using multiple kernels in the embedding layer is to allow the model to learn different shifts caused by deletion or insertion errors. Following this module, we sum over the cluster dimension with the goal of improving the model's robustness to differences in cluster size. This operator has the effect of non-coherent integration (NCI) and aims to increase the signal-to-noise ratio of the data.

After the NCI aligner layer, the model includes an embedding module whose architecture is similar to the alignment module; however, now the operations are employed on the whole cluster and not on each read independently. The goal is to learn correlations between the different reads and prepare the data for the transformer module. In addition, the embedding module outputs a sequence with the required output length and larger feature space.

The transformer module is a multihead transformer architecture, used with multilayer perceptron as feedforward layers[46]. We do not use position embedding in this module. After the last transformer block, a linear module is used to reduce the number of features to four, which represent one-hot encoding for the DNA representation.

The fusion layer is a vector of learnable parameters with a length of the required encoded sequence. This layer combines between the predictions of the two branches into a single prediction before a softmax operator, which transforms this representation to probabilities. Additional ablation studies and details on the architecture are provided in the Supplementary Information.

Loss function. To train our model a combination of cross entropy and consistency loss was used and are shown in the following equations:

$$\mathcal{L} = \lambda_{\text{ce}}\,\mathcal{L}_{\text{ce}} + \lambda_{\text{consistency}}\,\mathcal{L}_{\text{consistency}}, \tag{1}$$

$$\mathcal{L}_{\text{ce}} = -\frac{1}{n}\sum_{n} y_n \log(\Theta_n), \tag{2}$$

$$\mathcal{L}_{\text{consistency}} = 0.5 \times \left(\mathcal{L}_{\text{ce}}(\Theta_{\text{Left Branch}}) + \mathcal{L}_{\text{ce}(\Theta_{\text{Right Branch}})}\right), \tag{3}$$

where λ_i are hyperparameters, Θ_i are the model prediction probabilities and y_n are the labels. The left and right branches refer to the Siamese architecture. The formulation for consistency loss that yielded the best results is shown in equation (3) and aims to minimize the average prediction of the two branches, giving the model the freedom to adapt to changing noise characteristics along the sequence length.

Training details. Data generation and training was implemented in Pytorch, the optimizer used was Adam with $\beta_1 = 0.9$, $\beta_2 = 0.999$, batch size 64 and learning rate utilized cosine decay from 3.141×10^{-5} to 3.141×10^{-7}. A single A40 GPU was used during training and inference. Training took 50 epochs for the Illumina experiment and 180 epochs for the Nanopore experiment, each containing 1M clusters and an average number of 8 DNA reads per cluster.

CPL. The CPL algorithm is a second reconstruction step used on clusters with low confidence from the DNN output. The algorithm receives a cluster of reads and its goal is to predict their encoded sequence. Since the clusters that are sent to the CPL algorithm have low confidence, often the DNN predictions of these clusters are not correct. Therefore, we built the CPL to not use either the inference created by the DNN nor the prior knowledge of the sequencing and synthesis error rates. The algorithm works directly on the clusters as they were obtained by the clustering algorithm.

First, the CPL algorithm takes the first read from the cluster, and then uses dynamic programming to calculate the edit distance of the first read from any of the other reads in the cluster. This step is used to estimate the errors that occurred in the reads of the cluster. On the basis of this calculation, the algorithm creates vectors of edit operations (deletions, insertions and substitutions) that describe how to transform the first read to any of the other reads in the cluster using edit operations. These edit operations vectors represent estimations of the number of occurrences of each edit error in the cluster and their location with respect to the first read. Next, the algorithm creates a directed acyclic graph based on these estimations of the errors. The vertices in the graph are the symbols of the first read or the error events that were predicted in it based on the calculations. The edges in the graph connect any two vertices that correspond to symbols/errors that occur in adjacent locations. The weights of the edges are defined based on the occurrences of the two connected vertices in the edit operations vectors. Finally, the algorithm returns the longest path, which represents the sequence with maximum probability based on the algorithm's estimations.

Confidence filter and safety margin. One of the key features in our suggested solution is the confidence filter, which is a function that allows us to decide whether to rely on the DNN's output. The confidence filter is used for two purposes, the first is classifying highly erroneous DNN predictions as erasures, where in our pipeline this equates to a missing cluster. This is beneficial as correcting erasures requires less redundancy than correcting substitutions of entire sequences. The second purpose is to decide which of the clusters should be sent to the CPL algorithm.

The main component of the confidence filter is a confidence function that operates on the soft output of the DNN. The soft output of the DNN is a $4 \times L$ matrix M, in which the four entries of the ith column can be thought as the probabilities that the ith symbol of the prediction is the corresponding symbol ACGT and L is the encoded sequence length.

In the confidence function we utilize this property, specifically, we use the arithmetic mean of the maximal value in each column, which represents the probability that the predicted symbol is correct. This mean value is defined as $m(M) \triangleq \frac{1}{L}\sum_{j=1}^{L} \max\{M_{1,j}, M_{2,j}, M_{3,j}, M_{4,j}\}$ where $M_{i,j}$ is the value of the ith entry in the jth column of M. Intuitively, smaller values of $m(M)$ correspond to cases in which the DNN is less confident in its output.

An additional property that is considered in the confidence filter is the cluster size. More precisely, our confidence filter considers the fact that the output of the confidence function of smaller clusters, that is, lower signal-to-noise ratio, tends to be lower in general and integrates this property to the final filtering decision. The confidence function is defined as

$$c(M, \text{clustersize}) \triangleq m(M)^{2\times\text{clustersize}}.$$

The confidence filter evaluates the following conditions:

1. If $c(M, \text{clustersize}) \le \text{confidence}_{\text{threshold}}$ and $\text{clustersize} \le \text{clustersize}_{\text{threshold}}$, the cluster is classified as an erasure (that is, missing cluster).
2. Otherwise, if the CPL is incorporated in the pipeline,

$$c(M, \text{clustersize}) \le \text{confidence}_{\text{threshold}}$$
$$\text{and clustersize} > \text{clustersize}_{\text{threshold}}.$$

then the cluster is passed to the CPL algorithm.

3. Otherwise, the confidence filter trusts the DNN output and passes it as it is to the decoder.

The optimization for the accuracy versus run-time tradeoff is controlled by the values of $\text{confidence}_{\text{threshold}}$ and $\text{clustersize}_{\text{threshold}}$ and was performed using the pilot datasets. The performance of the DNN on the Illumina dataset was very good and did not require any other mechanism, therefore we set the $\text{confidence}_{\text{threshold}}$ as 0 and the $\text{clustersize}_{\text{threshold}}$ as 3. The performance of the DNN on the Nanopore single flowcell did require additional mechanisms to ensure successful information retrieval and a robust safety margin. Hence, we chose a safety margin of 1%, which yielded the values of a $\text{confidence}_{\text{threshold}}$ of 0.7 and a $\text{clustersize}_{\text{threshold}}$ of 4. Evaluation was performed on the test datasets with the details provided in the Supplementary Information.

Laboratory procedures. Before each sequencing, we performed PCR amplification using the standard protocol for Q5 High-Fidelity DNA polymerase (New England Biolabs, M0491S) for 12 cycles[47]. For Illumina sequencing, the paired-end reads were merged using the PEAR software tool[48]. Sequencing with the Oxford Nanopore MinION was done using the Oxford Nanopore's Ligation Sequencing Kit 110 with flowcell 9.4.1 (ref. 49).

Data availability

The datasets created and discussed in this Article are available via Zenodo at https://zenodo.org/records/13896773 (ref. 50). The synthetic data generator used to create the simulated training data is available in the data generator folder of our GitHub repository at https://github.com/itaiorr/Deep-DNA-based-storage.git (ref. 51). The publicly available datasets used in this study are sourced from the following publications: Grass et al.[13,52], available via Zenodo at https://zenodo.org/records/14290755 (ref. 53); Erlich et al.[15], available at http://www.ebi.ac.uk/ena/data/view/PRJEB19305 and http://www.ebi.ac.uk/ena/data/view/PRJEB19307; Organick et al.[16], available at http://misl.cs.washington.edu/data and https://github.com/uwmisl/data-nbt17; and Srinivasavaradhan et al.[7], available via GitHub at https://github.com/microsoft/clustered-nanopore-reads-dataset. The datasets from refs. 7,13,15, where the reads are classified into clusters by using our binning algorithm, are available via Zenodo at https://zenodo.org/records/14296588 (ref. 54). Binning of the dataset from ref. 16 is possible by applying the preprocessing and binning scripts (reads_preprocessor.py and binning.py) available via GitHub at https://github.com/itaiorr/Deep-DNA-based-storage.git (ref. 51). A parametric comparison of previous DNA data storage experiments[55–57], with additional parameters related to our data, is provided in Supplementary Table 4.

Code availability

The code used for this work is available via GitHub at https://github.com/itaiorr/Deep-DNA-based-storage.git (ref. 51).

References

1. Rydning, D. R. J. G. J., Reinsel, J. & Gantz, J. *The Digitization of the World from Edge to Core (International Data Corporation*, 2018).
2. Meiser, L. C. et al. Synthetic DNA applications in information technology. *Nat. Commun.* **13**, 352 (2022).
3. Ceze, L., Nivala, J. & Strauss, K. Molecular digital data storage using DNA. *Nat. Rev. Genet.* **20**, 456–466 (2019).
4. LeProust, E. M. et al. Synthesis of high-quality libraries of long (150 mer) oligonucleotides by a novel depurination controlled process. *Nucleic Acids Res.* **38**, 2522–2540 (2010).
5. Heckel, R., Mikutis, G. & Grass, R. N. A characterization of the DNA data storage channel. *Sci. Rep.* **9**, 9663 (2019).
6. Sabary, O., Yucovich, A., Shapira, G. & Yaakobi, E. Reconstruction algorithms for DNA storage systems. *Sci. Rep.* **14**, 951 (2024).
7. Srinivasavaradhan, S. R., Gopi, S., Pfister, H. D. & Yekhanin, S. Trellis BMA: coded trace reconstruction on IDS channels for DNA storage. In *2021 IEEE International Symposium on Information Theory (ISIT)* (ed. Dey, B.) 2453–2458 (IEEE, 2021); https://doi.org/10.1109/ISIT45174.2021.9517821
8. Lenz, A. et al. Concatenated codes for recovery from multiple reads of DNA sequences. In *2020 IEEE Information Theory Workshop (ITW)* (ed. Dalai, M.) 1–5 (IEEE, 2021).
9. Levenshtein, V. I. Efficient reconstruction of sequences from their subsequences or supersequences. *J. Comb. Theory. Ser. A* **93**, 310–332 (2001).
10. McGregor, A., Price, E. & Vorotnikova, S. Trace reconstruction revisited. In *European Symposium on Algorithms* (eds Schulz, A. S. & Wagner, D.) 689–700 (Springer, 2014).
11. Church, G. M., Gao, Y. & Kosuri, S. Next-generation digital information storage in DNA. *Science* **337**, 1628–1628 (2012).
12. Goldman, N. et al. Towards practical, high-capacity, low-maintenance information storage in synthesized DNA. *Nature* **494**, 77–80 (2013).
13. Grass, R. N., Heckel, R., Puddu, M., Paunescu, D. & Stark, W. J. Robust chemical preservation of digital information on DNA in silica with error-correcting codes. *Angew. Chem. Int. Ed.* **54**, 2552–2555, (2015).
14. MacWilliams, F. J. & Sloane, N. J. A. *The Theory of Error-Correcting Codes* Vol. 16 (Elsevier, 1997).
15. Erlich, Y. & Zielinski, D. DNA fountain enables a robust and efficient storage architecture. *Science* **355**, 950–954 (2017).
16. Organick, L. et al. Random access in large-scale DNA data storage. *Nat. Biotechnol.* **36**, 242–248 (2018).
17. Yazdi, S. M. H. T., Gabrys, R. & Milenkovic, O. Portable and error-free DNA-based data storage. *Sci. Rep.* **7**, 5011 (2017).
18. Wang, Y. et al. High capacity DNA data storage with variable-length oligonucleotides using repeat accumulate code and hybrid mapping. *J. Biol. Eng.* **13**, 1–11 (2019).
19. Chandak, S. et al. Overcoming high Nanopore basecaller error rates for DNA storage via basecaller-decoder integration and convolutional codes. In *ICASSP 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (ed. Pérez-Neira, A. I.) 8822–8826 (IEEE, 2020).
20. Anavy, L., Vaknin, I., Atar, O., Amit, R. & Yakhini, Z. Data storage in DNA with fewer synthesis cycles using composite DNA letters. *Nat. Biotechnol.* **37**, 1237 (2019).
21. Cheraghchi, M. & Ribeiro, J. An overview of capacity results for synchronization channels. *IEEE Trans. Inf. Theory* **67**, 3207–3232 (2020).
22. Qu, G., Yan, Z. & Wu, H. Clover: tree structure-based efficient DNA clustering for DNA-based data storage. *Brief. Bioinform.* **23**, bbac336 (2022).
23. Rashtchian, C. et al. Clustering billions of reads for DNA data storage. *Adv. Neural Inf. Process. Syst.* **30**, 3362–3373 (2017).
24. Viswanathan, K. & Swaminathan, R. Improved string reconstruction over insertion–deletion channels. In *Proc. 19th annual ACM–SIAM Symposium on Discrete Algorithms* (ed. Teng, S.-H.) 399–408 (Society for Industrial and Applied Mathematics, 2008).
25. Batu, T., Kannan, S., Khanna, S. & McGregor, A. Reconstructing strings from random traces. *SODA* **4**, 910–918 (2004).
26. Holden, N., Pemantle, R. & Peres, Y. Subpolynomial trace reconstruction for random strings and arbitrary deletion probability. In *Conference on Learning Theory* Vol. 75 (eds Bubeck, S. et al.) 1799–1840 (PMLR, 2018).
27. Holenstein, T., Mitzenmacher, M., Panigrahy, R. & Wieder, U. Trace reconstruction with constant deletion probability and related results. In *Proc. 19th Annual ACM–SIAM Symposium on Discrete Algorithms* (ed. Teng, S.-H.) 389–398 (Society for Industrial and Applied Mathematics, 2008).
28. Nazarov, F. & Peres, Y. Trace reconstruction with $\exp(O(n^{1/3}))$ samples. In *Proc. 49th Annual ACM SIGACT Symposium on Theory of Computing* (eds Hatami, H. & McKenzie, P.) 1042–1046 (ACM, 2017).
29. Peres, Y. & Zhai, A. Average-case reconstruction for the deletion channel: subpolynomially many traces suffice. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)* (ed. Umans, C.) 228–239 (IEEE Computer Society, 2017).
30. Bee, C. et al. Content-based similarity search in large-scale DNA data storage systems. *Nat. Commun.* **12**, 4764 (2021).
31. Pan, C. et al. Rewritable two-dimensional DNA-based data storage with machine learning reconstruction. *Nat. Commun.* **13**, 2984 (2022).
32. Wolf, J. On codes derivable from the tensor product of check matrices. *IEEE Trans. Inf. Theory* **11**, 281–284 (1965).

33. Sabary, O. et al. SOLQC: synthetic oligo library quality control tool. *Bioinformatics* **37**, 720–722 (2021).
34. *Preserving Our Digital Legacy: an Introduction to DNA Data Storage* (DNA Data Storage Alliance, 2021).
35. Gopalan, P. S. et al. Trace reconstruction from noisy polynucleotide sequencer reads. *US Patent Application* **15/536**, 115 (2018).
36. Marcus, B. H., Roth, R. M., & Siegel, P. H. Constrained systems and coding for recording channels. In *An Introduction to Coding for Constrained Systems* (eds Pless, V. & Huffman, W. C.) (Elsevier, 1998).
37. Gimpel, A. L., Stark, W. J., Heckel, R. & Grass, R. N. A digital twin for DNA data storage based on comprehensive quantification of errors and biases. *Nat. Commun.* **14**, 6026 (2023).
38. Bohlin, J., Rose, B. & Pettersson, J. H. O. Estimation of AT and GC content distributions of nucleotide substitution rates in bacterial core genomes. *Big Data Anal.* **4**, 1–11 (2019).
39. Weindel, F., Gimpel, A. L., Grass, R. N. & Heckel, R. Embracing errors is more effective than avoiding them through constrained coding for DNA data storage. In *2023 59th Annual Allerton Conference on Communication, Control, and Computing* 1–8 (IEEE, 2023).
40. Stoler, N. & Nekrutenko, A. Sequencing error profiles of Illumina sequencing instruments. *NAR Genom. Bioinform.* **3**, lqab019 (2021).
41. Ping, Z. et al. Towards practical and robust DNA-based data archiving using the yin-yang codec system. *Nat. Comput. Sci.* **2**, 234–242 (2022).
42. Chaykin, G., Furman, N., Sabary, O., Ben-Shabat, D. & Yaakobi, E. DNA-storalator: end-to-end DNA storage simulator. In *13th Annual Non-volatile Memories Workshop* (2022).
43. Xiao, T. et al. Early convolutions help transformers see better. *Adv. Neural Inf. Process. Syst.* **34**, 30392–30400 (2021).
44. Chowdhury, B. & Garai, G. A review on multiple sequence alignment from the perspective of genetic algorithm. *Genomics* **109**, 419–431 (2017).
45. Chollet, F. Xception: deep learning with depthwise separable convolutions. In *Proc. IEEE Conference on Computer Vision and Pattern Recognition* 1251–1258 (IEEE, 2017).
46. Vaswani, A. et al. Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30**, (2017).
47. Menin, J. F. & Nichols, N. M. *Multiplex PCR using Q5 High-Fidelity DNA Polymerase* (New England Biolabs, 2013).
48. Zhang, J., Kobert, K., Flouri, T. & Stamatakis, A. PEAR: a fast and accurate Illumina Paired-End reAd mergeR. *Bioinformatics* **30**, 614–620 (2014).
49. Wang, Y., Zhao, Y., Bollas, A., Wang, Y. & Au, K. F. Nanopore sequencing technology, bioinformatics and applications. *Nat. Biotechnol.* **39**, 1348–1365 (2021).
50. Bar-Lev, D., Orr, I., Sabary, O., Etzion, T. & Yaakobi, E. Datasets of scalable and robust DNA-based storage via coding theory and deep learning. *Zenodo* https://doi.org/10.5281/zenodo.13896773 (2024).
51. Bar-Lev, D., Orr, I., Sabary, O., Etzion, T. & Yaakobi, E. Code repository for scalable and robust DNA-based storage via coding theory and deep learning. *Zenodo* https://doi.org/10.5281/zenodo.14266018 (2024).
52. Grass, R. N., Heckel, R., Puddu, M., Paunescu, D. & Stark, W. J. Dataset for "Robust chemical preservation of digital information on DNA in silica with error-correcting codes". *Zenodo* https://doi.org/10.5281/zenodo.14290754 (2015).
53. Grass, R. N. et al. Dataset for "Robust chemical preservation of digital information on DNA in silica with error-correcting codes". *Zenodo* https://doi.org/10.5281/zenodo.14290755 (2015).
54. Bar-Lev, D., Orr, I., Sabary, O., Etzion, T. & Yaakobi, E. Prepared binned DNA data storage datasets for reconstruction benchmarking. *Zenodo* https://doi.org/10.5281/zenodo.14296588 (2024).
55. Bornholt, J. et al. A DNA-based archival storage system. In *Proc. 21st International Conference on Architectural Support for Programming Languages and Operating Systems* 637–649 (ACM, 2016).
56. Blawat, M. et al. Forward error correction for DNA data storage. *Procedia Comput. Sci.* **80**, 1011–1022 (2016).
57. Wagner, R. A. & Fischer, M. J. The string-to-string correction problem. *J. ACM* **21**, 168–173 (1974).

Acknowledgements

D.B.-L., I.O., O.S. and E.Y. were Funded by the European Union (ERC, DNAStorage, 101045114 and EIC, DiDAX 101115134). The views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them. D.B.-L. and T.E. were supported in part by ISF grant no. 222/19. The authors thank S. Goldberg, N. Kikuchi and R. Amit for assistance with the wet experiments and providing the lab and equipment; N. Fourier and L. Linde for their help in the sequencing processes and D. B. Shabat for his assistance and advice. Finally, we thank A. Yucovich for his help with developing the CPL algorithm.

Author contributions

Conceptualization: D.B.-L., I.O. and O.S. Methodology: D.B.-L., I.O. and O.S. Investigation: D.B.-L., I.O., O.S., T.E. and E.Y. Supervision: T.E. and E.Y. Writing, review and editing: D.B.-L., I.O., O.S., T.E. and E.Y.

Competing interests

D.B.-L., I.O., O.S., T.E. and E.Y. are inventors on patent application US 18/233,855 submitted by the Technion–Israel Institute of Technology and Bar Ilan University.

Additional information

Supplementary information The online version contains supplementary material available at https://doi.org/10.1038/s42256-025-01003-z.

Correspondence and requests for materials should be addressed to Daniella Bar-Lev, Itai Orr or Omer Sabary.

Peer review information *Nature Machine Intelligence* thanks the anonymous reviewer(s) for their contribution to the peer review of this work.